



MIDDLE EAST TECHNICAL UNIVERSITY

MMI714

Generative Models for Multimedia

Diffusion-based Generative Models

Welcome

This is MMI714

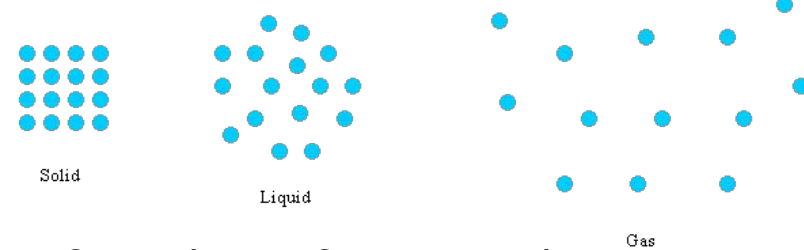
“Generative Models for Multimedia”

This week we will talk about:

Diffusion Models,
Denoising DMs, Score-based GMs,
Stochastic DEs, Efficient Sampling

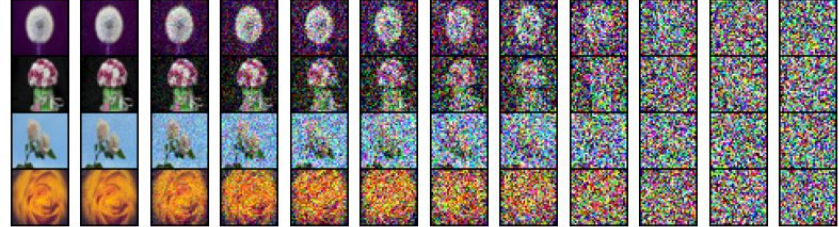


Diffusion

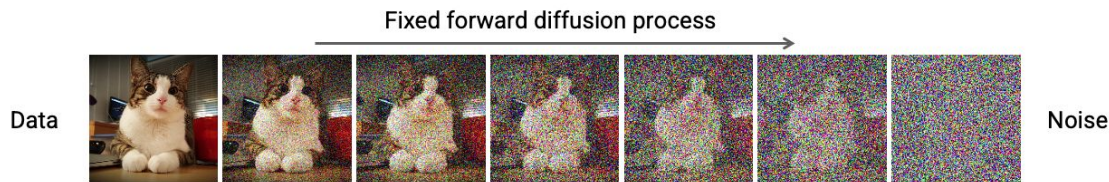


- In physics, “Diffusion” is the net movement of anything (for example, atoms, ions, molecules, energy) generally from a region of higher concentration to a region of lower concentration [Wiki].
- As particles diffuse from an area of higher concentration to an area of lower concentration, the overall randomness or disorder in the system increases.
- The distribution becomes more uniform, and entropy tends to increase.

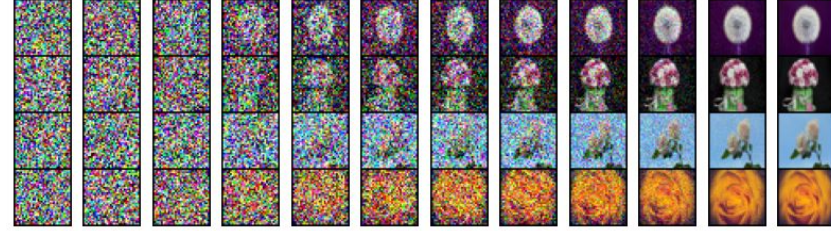
Diffusion in Inf. Theory



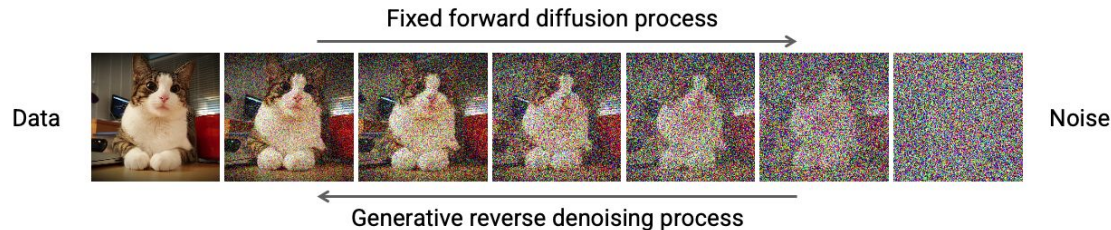
- In our context, Diffusion refers to the process of adding controlled noise or perturbations to an image to generate more diverse or complex samples.
- This is the "forward" direction of the diffusion process, where you start with a clean or noise-free image and gradually introduce noise.
- From low entropy to high entropy!



Reverse Diffusion

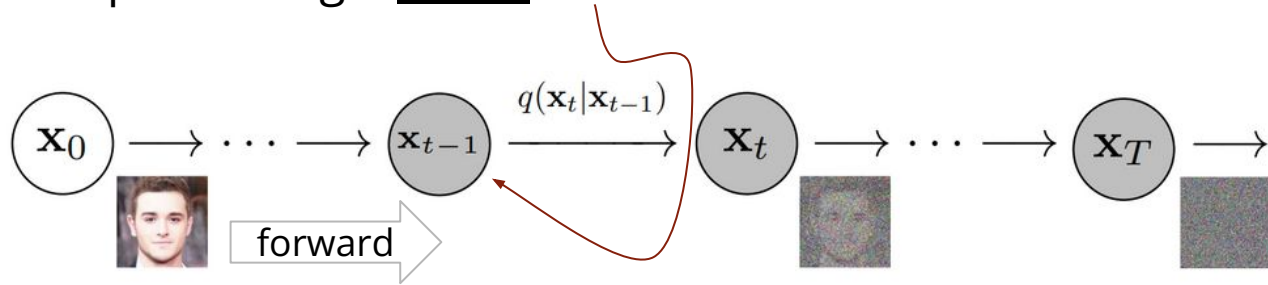


- It's the inverse of the forward diffusion process.
- The reverse diffusion process in generative diffusion models refers to the procedure of reconstructing a clean or noise-free image from a given noisy or perturbed input.



Diffusion Models

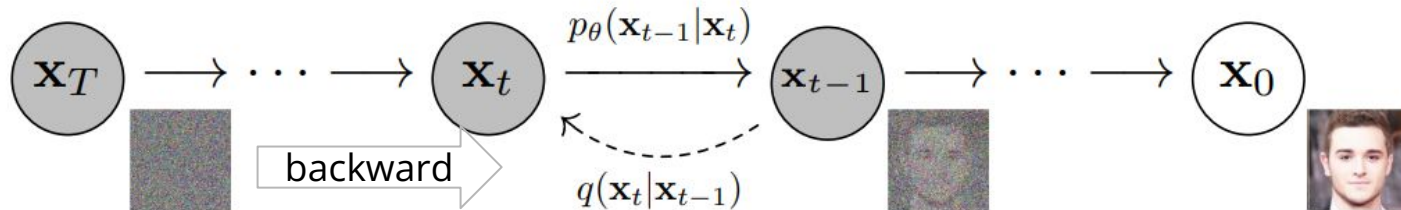
- DMs work by destroying training data through the successive addition of Gaussian noise, and then learning to recover the data by reversing this noising process.
- More specifically, a Diffusion Model is a latent variable model which maps to the latent space using a **fixed** Markov chain.



" $q()$ is **fixed**" : same NN is used at each step

Diffusion Models

- For sampling we need to learn the reverse process. Because the latent space is the noise.
- In forward process, the image is asymptotically transformed to pure Gaussian noise.
- The goal of training a diffusion model is to learn the reverse process - i.e. training .



again $p()$ "fixed" : same NN is used/learned at each step, which define the entire reverse process

Diffusion Models

- There are three predominant formulations:
 - **Denoising Diffusion Probabilistic Models (DDPMs)**
 - **Score-based Generative Models (SGMs)**
 - Stochastic Differential Equations (SDEs)

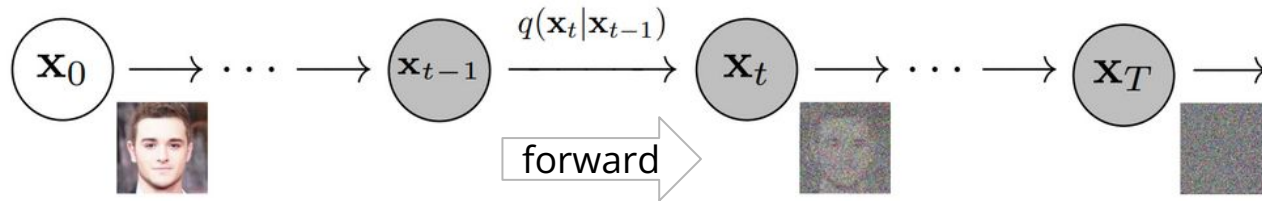
Denoising Diffusion Probabilistic Models

- This is the basic one, which you were introduced in the previous slides.
- A denoising diffusion probabilistic model (DDPM) makes use of two Markov chains:
 - a forward chain that perturbs data to noise,
 - and a reverse chain that converts noise back to data.
- The former is typically hand-designed with the goal to transform any data distribution into a simple prior distribution (e.g., standard Gaussian),
 - We do not care about this one. It is simple.
- While the latter Markov chain reverses the former by learning transition kernels parameterized by deep neural networks.
 - This is the one we “learn”.

DDPM Forward

- Given a data distribution $x_0 \sim q(x_0)$, the forward Markov process generates a sequence of random variables $x_1, x_2 \dots x_T$ with **transition kernel** $q(x_t | x_{t-1})$.
- Using the chain rule of probability and the Markov property, we can factorize the joint distribution of $x_1, x_2 \dots x_T$ conditioned on x_0 , denoted as $q(x_1, \dots, x_T | x_0)$, into

$$q(x_1, \dots, x_T | x_0) = \prod_{t=1}^T q(x_t | x_{t-1}). \quad (1)$$



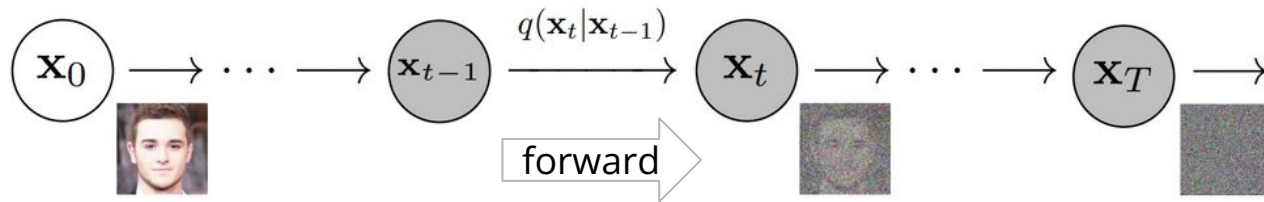
DDPM Forward

- The typical design for the transition kernel is Gaussian, and the most common choice for the transition kernel is:

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}), \quad (2)$$

where $\beta_t \in (0, 1)$ is a hyperparameter chosen ahead of model training.

(although other types of kernels are also applicable.)



DDPM Forward

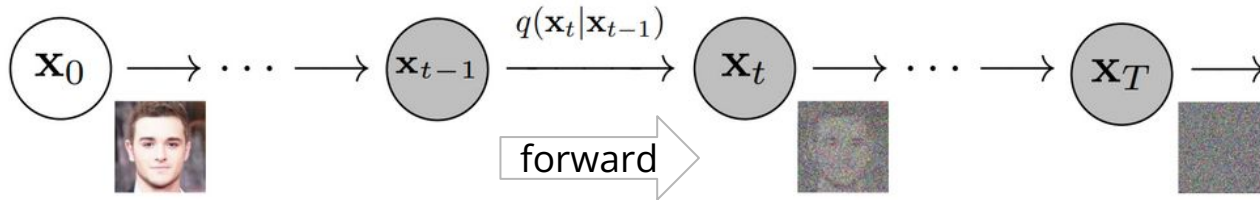
- The typical design for the transition kernel is Gaussian, and the most common choice for the transition kernel is:

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}), \quad (2)$$

where $\beta_t \in (0, 1)$ is a hyperparameter chosen ahead of model training.

β is the level of diffusion at time step t .

- If 0 $\rightarrow$ no diffusion, no noise added
- If 1 $\rightarrow$ complete jump to infinite entropy (noise) (we don't want both)



$$q(\mathbf{x}_1, \dots, \mathbf{x}_T | \mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t | \mathbf{x}_{t-1}). \quad (1)$$

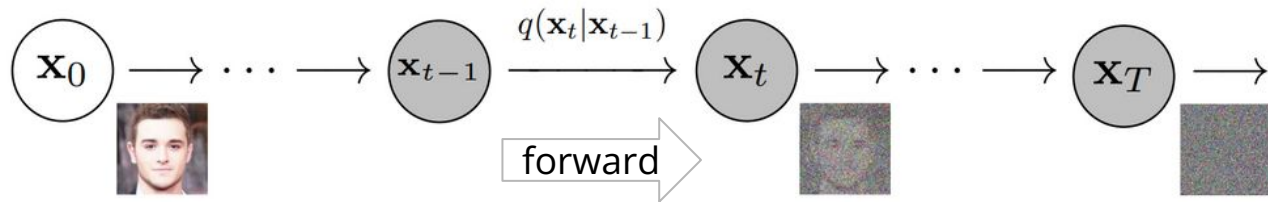
$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}), \quad (2)$$

DDPM Forward

- When we combine (1) and (2) and using the marginalisation introduced in the original DM paper by [Sohl-Dickstein et al. \(2015\)](#) because $\beta_t \in (0, 1)$ after enough time steps, $\mathbf{x}_T$ will become Gaussian:

$$q(\mathbf{x}_T) := \int q(\mathbf{x}_T | \mathbf{x}_0) q(\mathbf{x}_0) d\mathbf{x}_0 \approx \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I}).$$

This is simply how we manifest the forward diffusion process mathematically.

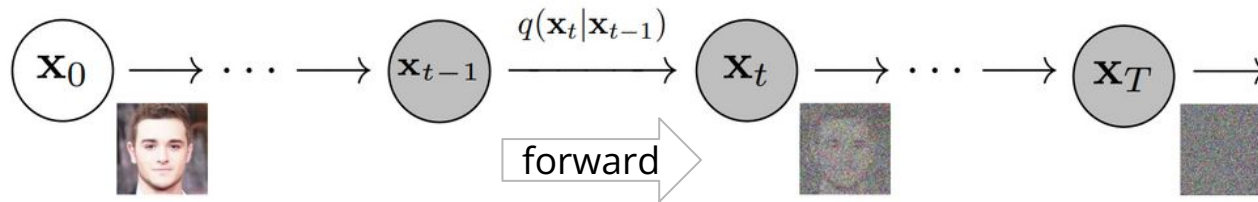


$$q(\mathbf{x}_1, \dots, \mathbf{x}_T \mid \mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t \mid \mathbf{x}_{t-1}). \quad (1)$$

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}), \quad (2)$$

DDPM Forward

- Question?: Why not make $\beta=1$ and jump in a single step?

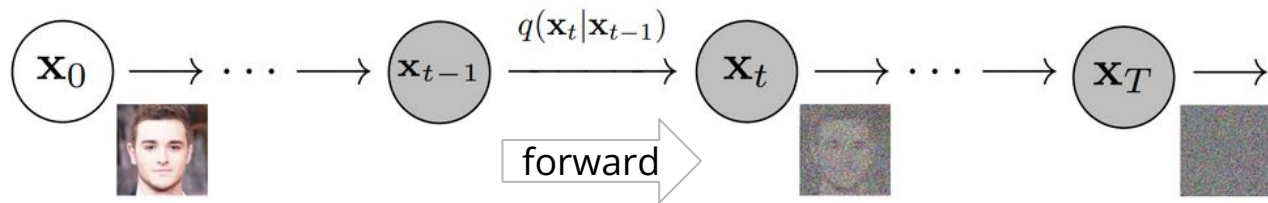


$$q(\mathbf{x}_1, \dots, \mathbf{x}_T | \mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t | \mathbf{x}_{t-1}). \quad (1)$$

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}), \quad (2)$$

DDPM Forward

- Question?: Why not make $\beta=1$ and jump in a single step?
- Answer: You may think that each intermediate step contributes to the overall loss function during training (analogous to keeping activations during the forward propagation in deep neural networks), which would be logical.

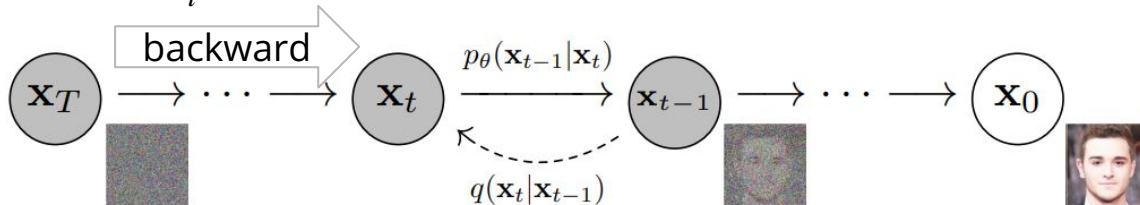


DDPM Backward

- DDPMs start by first generating an unstructured noise vector from the prior distribution (this was forward),
- (now backward) then gradually remove noise therein by running a learnable Markov chain in the reverse time direction.
- The learnable transition kernel $p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t)$ takes the form:

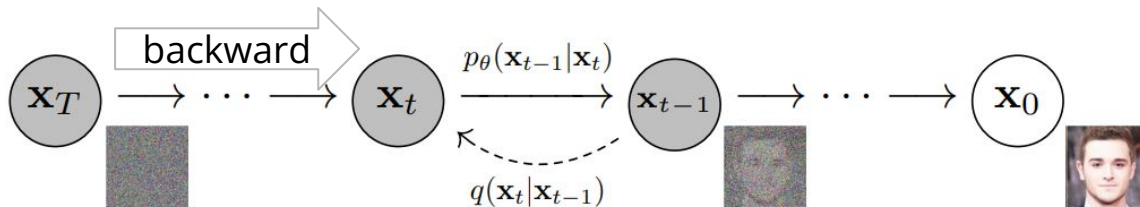
$$p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_{\theta}(\mathbf{x}_t, t), \Sigma_{\theta}(\mathbf{x}_t, t))$$

- where θ denotes model parameters, and the mean $\mu_{\theta}(\mathbf{x}_t, t)$ and variance $\Sigma_{\theta}(\mathbf{x}_t, t)$ are parameterized by deep neural networks.



DDPM Backward

- Do we explicitly learn $\mu_\theta(\mathbf{x}_t, t)$ and $\Sigma_\theta(\mathbf{x}_t, t)$
 - Not necessarily
- So how to train?
 - By making the reverse Markov chain to match the actual time reversal of the forward Markov chain.
- That is, we have to adjust the parameter θ so that the joint distribution of the reverse Markov chain $p_\theta(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T) := p(\mathbf{x}_T) \prod_{t=1}^T p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)$ approximates that of the (reverse of the) forward process. $q(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T) := q(\mathbf{x}_0) \prod_{t=1}^T q(\mathbf{x}_t | \mathbf{x}_{t-1})$

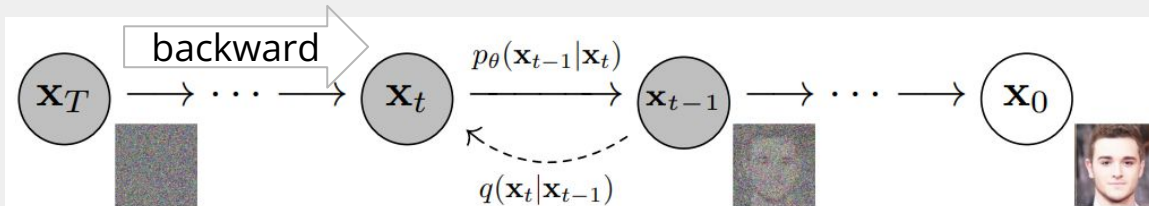


$$D_{\text{KL}}(Q \parallel P) = \sum_{x \in \mathcal{X}} Q(x) \ln \left(\frac{Q(x)}{P(x)} \right)$$

DDPM Backward

- We achieve this by minimizing the Kullback-Leibler (KL) divergence:

$$\text{KL}(q(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T) \parallel p_{\theta}(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T))$$



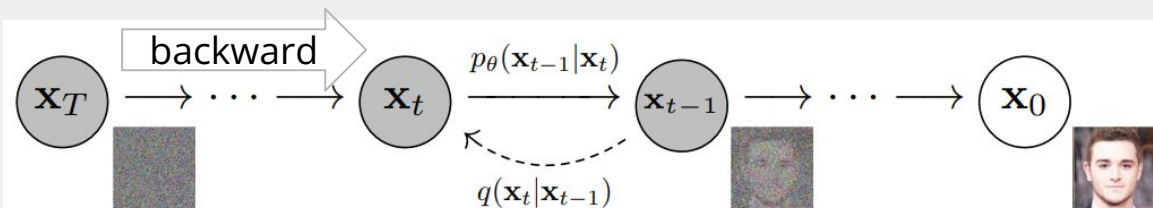
DDPM Backward

- We achieve this by minimizing the Kullback-Leibler (KL) divergence:
 - (i) comes easily from KL definition:

$$\mathbb{E}_{p(\mathbf{x}; \theta)}[f(\mathbf{x}; \phi)] = \int p(\mathbf{x}; \theta) f(\mathbf{x}; \phi) d\mathbf{x}$$

$$\text{KL}(q(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T) \parallel p_\theta(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T))$$

$$\stackrel{(i)}{=} -\mathbb{E}_{q(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T)} [\log p_\theta(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T)] + \text{const}$$



DDPM Backward

- We achieve this by minimizing the Kullback-Leibler (KL) divergence:

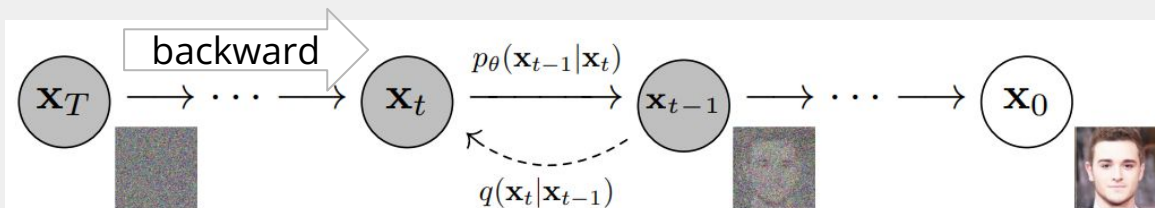
- (i) comes easily from KL definition:
- (ii) comes from the fact that q and p are both products of distributions:

$$p_{\theta}(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T) := p(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$$

$$q(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T) := q(\mathbf{x}_0) \prod_{t=1}^T q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$$

?

$$\begin{aligned} & \text{KL}(q(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T) \parallel p_{\theta}(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T)) \\ & \stackrel{(i)}{=} -\mathbb{E}_{q(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T)} [\log p_{\theta}(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T)] + \text{const} \\ & \stackrel{(ii)}{=} \underbrace{\mathbb{E}_{q(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T)} \left[-\log p(\mathbf{x}_T) - \sum_{t=1}^T \log \frac{p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t)}{q(\mathbf{x}_t \mid \mathbf{x}_{t-1})} \right]}_{:= -L_{\text{VLB}}(\mathbf{x}_0)} + \text{const} \end{aligned}$$



DDPM Backward

- We achieve this by minimizing the Kullback-Leibler (KL) divergence:

- (i) comes easily from KL definition:
- (ii) comes from the fact that q and p are both products of distributions:

$$p_{\theta}(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T) := p(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$$

$$q(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T) := q(\mathbf{x}_0) \prod_{t=1}^T q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$$

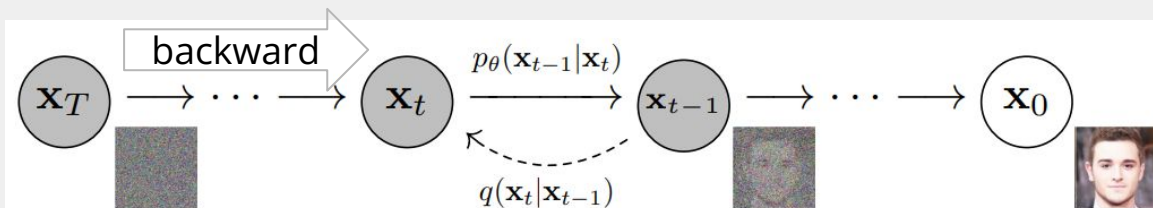
- (iii) comes from Jensen's inequality

$$\text{KL}(q(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T) \parallel p_{\theta}(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T))$$

$$\stackrel{(i)}{=} -\mathbb{E}_{q(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T)} [\log p_{\theta}(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T)] + \text{const}$$

$$\stackrel{(ii)}{=} \underbrace{\mathbb{E}_{q(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T)} \left[-\log p(\mathbf{x}_T) - \sum_{t=1}^T \log \frac{p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t)}{q(\mathbf{x}_t \mid \mathbf{x}_{t-1})} \right]}_{:= -L_{\text{VLB}}(\mathbf{x}_0)} + \text{const}$$

$$\stackrel{(iii)}{\geq} \mathbb{E} [-\log p_{\theta}(\mathbf{x}_0)] + \text{const},$$



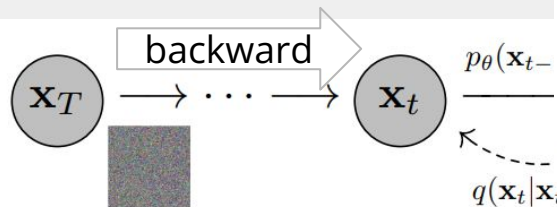
DDPM Backward

- We achieve this by
 - (i) comes easily from
 - (ii) comes from the f both products of dis

$$p_{\theta}(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T) := p(\mathbf{x}_T$$

$$q(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T) := q(\mathbf{x}_0$$

- (iii) comes from Jens



Jensen's Inequality: Intuition

$$\mathbb{E}[g(x)] \geq g[\mathbb{E}(x)] \text{ if } g \text{ is convex}$$

$g: x \rightarrow x^2$

$P(x=x)$	x	x^2
1/6	1	1
	2	4
	3	9
	4	16
	5	25
	6	36

The intuition behind Jensen's Inequality



Ox educ
17.1K subscribers

Subscribe

496



Share



Download



Clip



Save

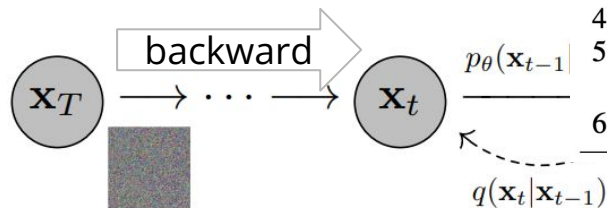


DDPM Backward

- We achieve this by minimizing the Kullback-Leibler (KL) divergence:
 - After derivations and derivations, the likelihood function becomes a sum of KL divergences:

$$D_{\text{KL}}(p_T(x_T) \| q(x_T | x_0)) + \sum_{t=2}^T D_{\text{KL}}(p(x_{t-1} | x_t; \theta) \| q(x_{t-1} | x_t, x_0)) + \int dx_1 dx_0 q(x_1, x_0) \ln p(x_0 | x_1)$$

- The process has still been general up until this point. But because both $p(x)$ and $q(x)$ are selected as Gaussians, the KL divergences become further simplified into:



Algorithm 1 Training

```

1: repeat
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ 
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$ 
4:    $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
5:   Take gradient descent step on
        $\nabla_{\theta} \| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \|^2$ 
6: until converged
  
```

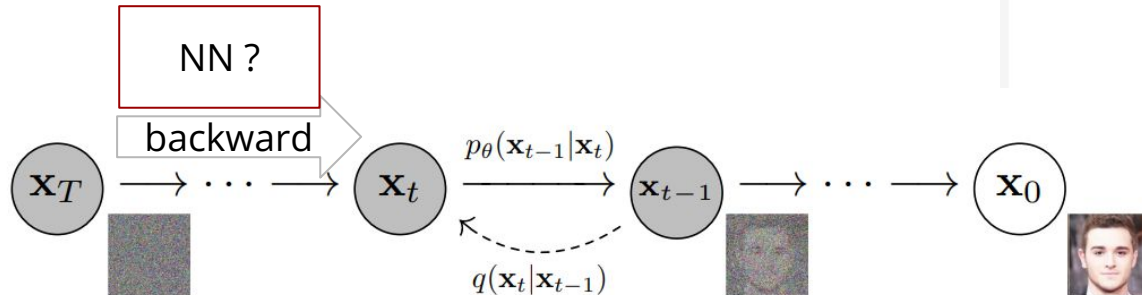
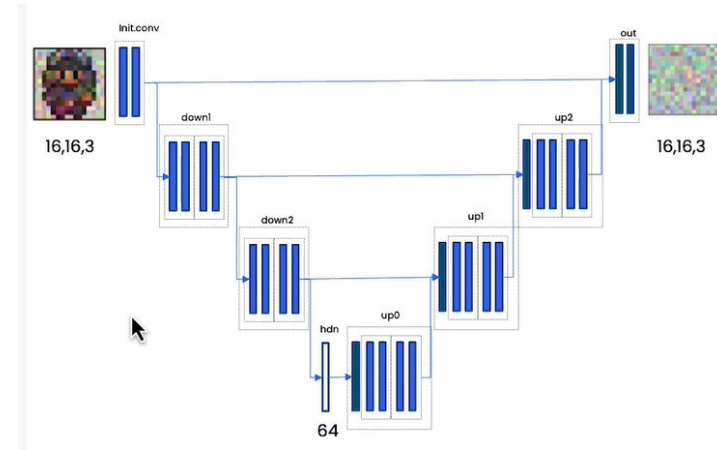
Algorithm 2 Sampling

```

1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $t = T, \dots, 1$  do
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$ 
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 
5: end for
6: return  $\mathbf{x}_0$ 
  
```

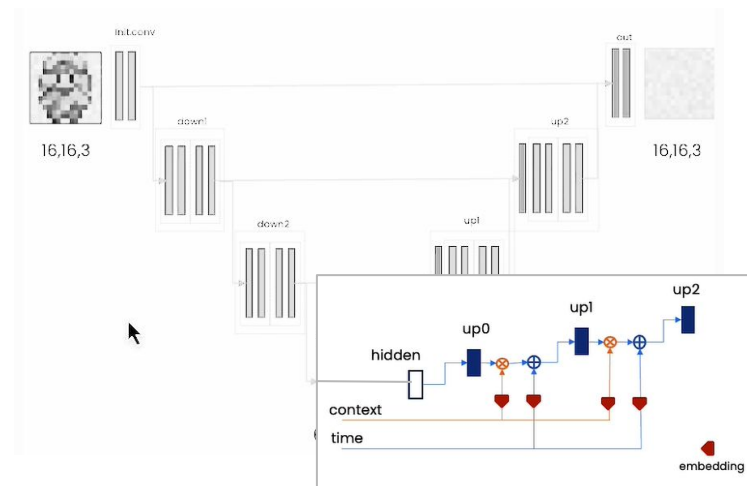
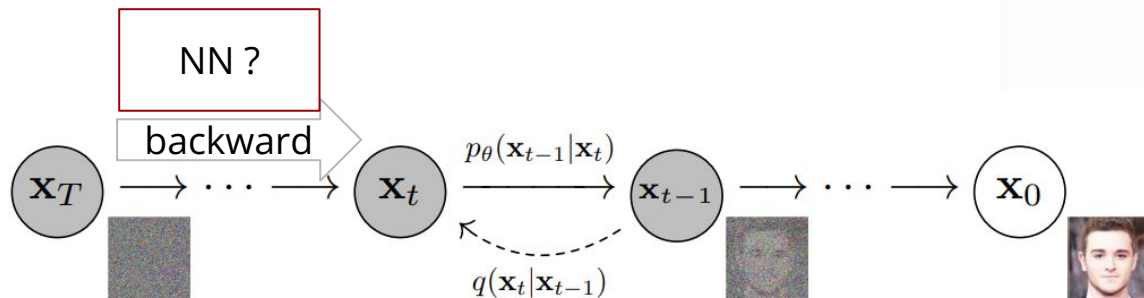
DDPM Neural Network

- What can we choose as the neural net that denoises the image?
- It could any architecture that can denoise data. U-Net for example, can output same size as input.
- That's why denoising, semantic segmentation, etc architectures can all be applied.



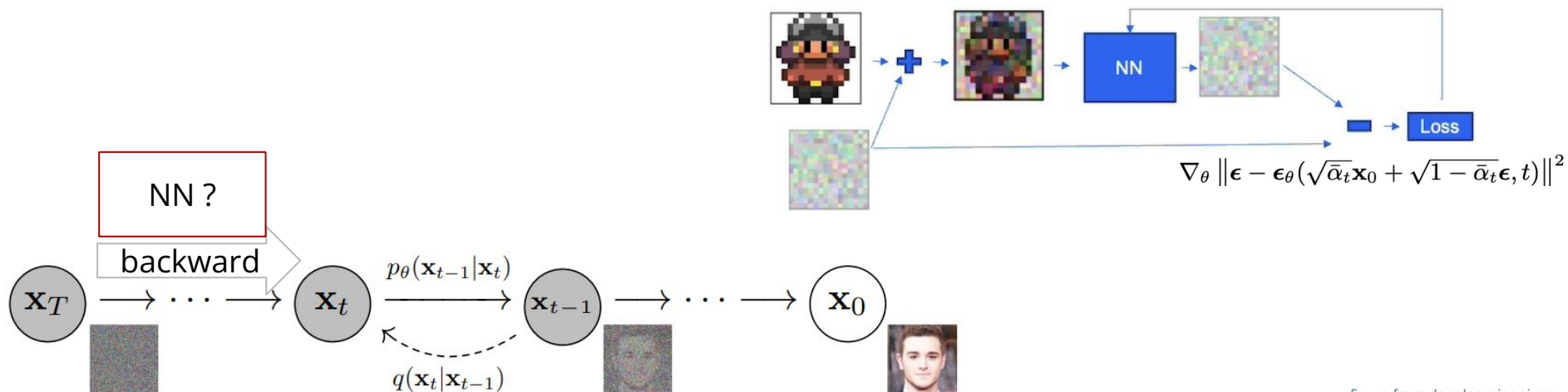
DDPM Neural Network

- Nice thing about U-Net is, it is very easy to embed additional information (conditions) through the decoder.
- Time information (noise scheduling) is done this way. But conditions can also be.



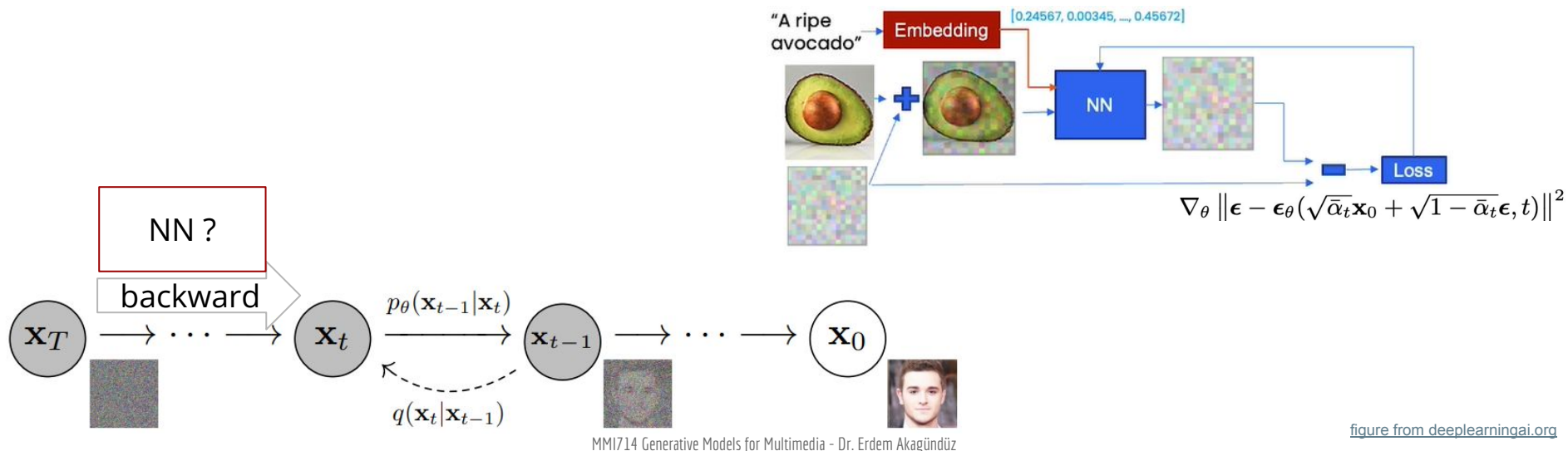
DDPM Neural Network

- The goal of the neural network is to predict noise.
- forward propagation:
 - We add noise to the sample,
 - We ask the neural network to predict that noise, and calculate a loss.



DDPM Neural Network

- And while you are doing this, you can easily embed any information into the network.





Score-based Diffusion

SCORE-BASED GENERATIVE MODELING THROUGH STOCHASTIC DIFFERENTIAL EQUATIONS

Yang Song*

Stanford University

yangsong@cs.stanford.edu

Jascha Sohl-Dickstein

Google Brain

jaschasd@google.com

Diederik P. Kingma

Google Brain

durk@google.com

Abhishek Kumar

Google Brain

abhishk@google.com

Stefano Ermon

Stanford University

ermon@cs.stanford.edu

Ben Poole

Google Brain

pooleb@google.com

- Some mathematical background:
 - Likelihood vs implicit models
 - Log-likelihood optimization and normalization issue
 - How do VAEs, NF, DDPM handle this?
 - What is a score function and why?
 - Score function for parameters vs data?
 - Fischer's Information Matrix
 - Sampling: Langevin Dynamics

<https://arxiv.org/abs/2011.13456>

ABSTRACT

Creating noise from data is easy; creating data from noise is generative modeling. We present a stochastic differential equation (SDE) that smoothly transforms a complex data distribution to a known prior distribution by slowly injecting noise, and a corresponding reverse-time SDE that transforms the prior distribution back into the data distribution by slowly removing the noise. Crucially, the reverse-time SDE depends only on the time-dependent gradient field (a.k.a., score) of the perturbed data distribution. By leveraging advances in score-based generative modeling, we can accurately estimate these scores with neural networks, and use numerical SDE solvers to generate samples. We show that this framework encapsulates previous approaches in score-based generative modeling and diffusion probabilistic modeling, allowing for new sampling procedures and new modeling capabilities. In particular, we introduce a predictor-corrector framework to correct errors in the evolution of the discretized reverse-time SDE. We also derive an equivalent neural ODE that samples from the same distribution as the SDE, but additionally enables exact likelihood computation, and improved sampling efficiency. In addition, we provide a new way to solve inverse problems with score-based models, as demonstrated with experiments on class-conditional generation, image inpainting, and colorization. Combined with multiple architectural improvements, we achieve

Score-based Diffusion

Likelihood-based (Explicit) vs Implicit Generative Models

- Existing generative modeling techniques can largely be grouped into two categories based on how they represent probability distributions.
 - Likelihood-based models: which directly learn the distribution's probability density (or mass) function via (approximate) maximum likelihood. Typical likelihood-based models include autoregressive models , normalizing flow models, energy-based models (EBMs), and variational auto-encoders (VAEs)
 - Implicit generative models: where the probability distribution is implicitly represented by a model of its sampling process. The most prominent example is generative adversarial networks (GANs), where new samples from the data distribution are synthesized by transforming a random Gaussian vector with a neural network.

Score-based Diffusion

Likelihood-based (Explicit) vs Implicit Generative Models

- Likelihood-based models and implicit generative models, however, both have significant limitations:
 - Likelihood-based models either require strong restrictions on the model architecture to ensure a tractable normalizing constant for likelihood computation, or must rely on surrogate objectives to approximate maximum likelihood training.
 - Implicit generative models, on the other hand, often require adversarial training, which is notoriously unstable and can lead to mode collapse

Score-based Diffusion

Normalizing constant in likelihood optimization

- The issue of probability density function (PDF) normalization is a common challenge in likelihood-based generative models.
 - because a valid probability distribution must integrate to 1 over its entire domain.

- A PDF $p(x)$ describes the likelihood of observing a particular value x from a random variable. For $p(x)$ to be a valid probability distribution, it must satisfy two properties:
 - Non-negativity: $p(x) \geq 0$ for all x .
 - Normalization: $\int p(x) dx = 1$ over the entire domain.

Score-based Diffusion

Normalizing constant in likelihood optimization

- The issue of probability density function (PDF) normalization is a common challenge in likelihood-based generative models.
 - because a valid probability distribution must integrate to 1 over its entire domain.

- In generative modeling, the goal is often to learn a model $p_{\theta}(x)$ with parameters θ that captures the data distribution. This is typically done by maximizing the likelihood of the observed data under the model, leading to a likelihood-based loss function.
- The likelihood function $L(\theta) = p_{\theta}(x_1) \cdot p_{\theta}(x_2) \cdot \dots \cdot p_{\theta}(x_n)$ expresses how probable the observed data is under the model.

Score-based Diffusion

Normalizing constant in likelihood optimization

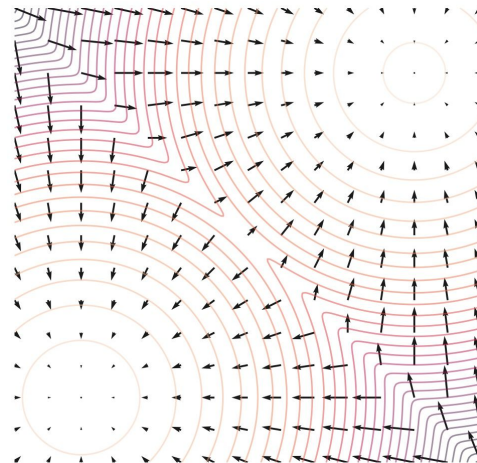
- The issue of probability density function (PDF) normalization is a common challenge in likelihood-based generative models.
 - because a valid probability distribution must integrate to 1 over its entire domain.

- Likelihood-based loss functions, such as the negative log-likelihood (NLL), are commonly used in training generative models. The NLL is derived from the negative logarithm of the likelihood function.
- The normalization constant is present in the likelihood function, and when computing the NLL, the log of this constant is subtracted. In practice, the normalization constant may be challenging to compute, but it cancels out when taking gradients for optimization.

Score-based Diffusion

Score-based diffusion models addresses this normalization problem.

- The key idea is to model the gradient of the log probability density function, a quantity often known as the (Stein) score function. How?
 - The gradient of a distribution forms a vector field.
 - If you have a probability distribution $p(x)$, and you take the gradient with respect to the parameters or variables x , the result is a vector of partial derivatives.
 - Each component of this vector corresponds to the rate of change of the probability with respect to a specific variable or parameter.



Score-based Diffusion

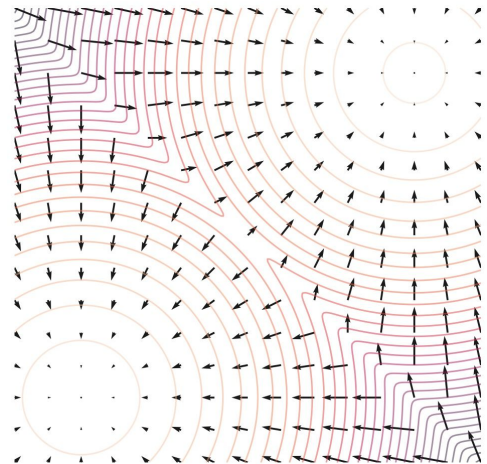
parameters θ or variable x ?

Score Function for Parameters (θ) - Fisher Score:

- When you take the gradient of the log-likelihood with respect to the parameters (θ), you get the Fisher score. This is often denoted as $\nabla_{\theta} \log p(x; \theta)$.
- The Fisher information matrix is the covariance of this score. It represents the sensitivity of the likelihood function to changes in the parameters.

Score Function for Data (x) - Score-Based Diffusion Models:

- In score-based models, the score function is usually with respect to the data (x), not the parameters. It's denoted as $\nabla_x \log p(x)$.
- The score-based models often use this score function to define a diffusion process for generating samples. The gradient points in the direction where the data likelihood is increasing.



Score-based Diffusion

Score-based diffusion models addresses this normalization problem.

- So because we are checking the gradient, score-based models are not required to have a tractable normalizing constant, and can be directly learned by score matching.
- Instead of maximizing the log-likelihood of the data:

$$\max_{\theta} \sum_{i=1}^N \log p_{\theta}(\mathbf{x}_i).$$

- We define an unnormalized probability model $f(\mathbf{x})$, which is like an unnormalized **score** of how likely the data is

Score-based Diffusion

- We define the relation between $p(\mathbf{x})$ and $f(\mathbf{x})$ to be in the form:

$$p_{\theta}(\mathbf{x}) = \frac{e^{-f_{\theta}(\mathbf{x})}}{Z_{\theta}},$$

where Z_{θ} is the normalizing constant

- WHY this form?
 - To make our life easier (i.e. you will see in a few slides)

When we make the assumption that the distribution $p(x)$ can be represented as $\frac{1}{Z} \exp(-f(x))$, we are essentially using a form that is quite flexible and general. The key aspect here is the exponential term $\exp(-f(x))$. Exponential functions have nice mathematical properties, and they can represent a wide range of shapes and behaviors.

Score-based Diffusion

$$\max_{\theta} \sum_{i=1}^N \log p_{\theta}(\mathbf{x}_i).$$

Here is the trick!



- The likelihood to be optimized requires θ to be a normalized probability density function.
- This is undesirable because in order to compute $p(\mathbf{x})$, we must evaluate the normalizing constant Z_{θ} - a typically intractable quantity for any general $f(\mathbf{x})$.
- Thus to make maximum likelihood training feasible, likelihood-based models must either
 - restrict their model architectures (e.g., causal convolutions in autoregressive models, invertible networks in normalizing flow models) to make tractable,
 - or approximate the normalizing constant (e.g., variational inference in VAEs)

Score-based Diffusion

Here is the trick!

- By modeling the gradient of the likelihood, a.k.a the “score function”, instead of the density function, we can sidestep the difficulty of intractable normalizing constants.
- The score function of a distribution is defined as:

$$\nabla_{\mathbf{x}} \log p(\mathbf{x}),$$

- This is the score function w.r.t to the variable $\mathbf{x}$, not the parameters θ

Score-based Diffusion

Here is the trick!

- Combining these two: $\nabla_{\mathbf{x}} \log p(\mathbf{x}), \quad p_{\theta}(\mathbf{x}) = \frac{e^{-f_{\theta}(\mathbf{x})}}{Z_{\theta}},$

$$s_{\theta}(\mathbf{x}) = \nabla_{\mathbf{x}} \log p_{\theta}(\mathbf{x}) = -\nabla_{\mathbf{x}} f_{\theta}(\mathbf{x}) - \underbrace{\nabla_{\mathbf{x}} \log Z_{\theta}}_{=0} = -\nabla_{\mathbf{x}} f_{\theta}(\mathbf{x}).$$

the score-based model $s_{\theta}(\mathbf{x})$ is independent of the normalizing constant Z_{θ} . This significantly expands the family of models that we can tractably use, since we don't need any special architectures to make the normalizing constant tractable.



$$D_F(p||q) = E_p \left[\left(\frac{\partial \log p(x)}{\partial x} - \frac{\partial \log q(x)}{\partial x} \right)^2 \right]$$

Score-based Diffusion

- Similar to likelihood-based models, we can train score-based models by minimizing the **Fisher divergence** (not KL not other, for a reason!) between the model and the data distributions, defined as:

$$\mathbb{E}_{p(\mathbf{x})} [\| \nabla_{\mathbf{x}} \log p(\mathbf{x}) - \mathbf{s}_{\theta}(\mathbf{x}) \|_2^2]$$

same

Indicates L2 norm (Fischer Div.)

- But how to calculate $\nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$ from the dataset?
- The main problem here is that we do not know **the data score** since it depends on knowing what $p_{\text{data}}(\mathbf{x})$ is.

Score-based Diffusion

- So, there exists a family of methods called score matching that minimize the loss function without knowledge of the ground-truth data score.

$$\mathbb{E}_{p_{\text{data}}(\mathbf{x})} [\|\mathbf{s}_{\theta}(\mathbf{x}) - \nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})\|_2^2]$$

Estimation of Non-Normalized Statistical Models by Score Matching

Aapo Hyvärinen

Helsinki Institute for Information Technology (BRU)

Department of Computer Science

FIN-00014 University of Helsinki, Finland

A.APO.HYVARINEN@HELUNKI.FI

Editor: Peter Dayan

Abstract

One often wants to estimate statistical models where the probability density function is known only up to a multiplicative normalization constant. Typically, one then has to resort to Markov Chain Monte Carlo methods, or approximations of the normalization constant. Here, we propose that such models can be estimated by minimizing the expected squared distance between the gradient of the log-density given by the model and the gradient of the log-density of the observed data. While the estimation of the gradient of log-density function is, in principle, a very difficult non-parametric problem, we prove a surprising result that gives a simple formula for this objective function. The density function of the observed data does not appear in this formula, which simplifies to a sample average of a sum of some derivatives of the log-density given by the model. The validity of the method is demonstrated on multivariate Gaussian and independent component analysis models, and by estimating an overcomplete filter set for natural image data.

Keywords: statistical estimation, non-normalized densities, pseudo-likelihood, Markov chain Monte Carlo, contrastive divergence

Score-based Diffusion

- Similar to likelihood-based models, we can train score-based models by minimizing the Fisher divergence between the model and the data distributions, defined as:

$$\mathbb{E}_{p_{\text{data}}(\mathbf{x})} [\|\mathbf{s}_{\theta}(\mathbf{x}) - \nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})\|_2^2]$$

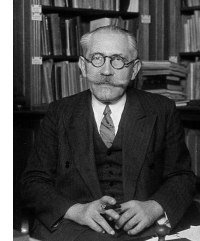
- Hyvärinen (2005) showed that optimizing the above loss function is equivalent optimizing the equation below:

$$\mathbb{E}_{p_{\text{data}}(\mathbf{x})} \left[\text{tr} (\nabla_{\mathbf{x}} \mathbf{s}_{\theta}(\mathbf{x})) + \frac{1}{2} \|\mathbf{s}_{\theta}(\mathbf{x})\|_2^2 \right]$$

$$\frac{dx}{dt} = \frac{1}{2} \nabla \log p(x) + \sqrt{\epsilon} \eta(t)$$

Here, x is the state of the system, $\nabla \log p(x)$ is the score function, ϵ is a small step size, and $\eta(t)$ is a Gaussian noise term at each time step.

“There are days when I wish I did not have to wear a tie to work every day.”
Paul Langevin



Score-based Sampling

- So we have a score function $s(x)$ which approximates the gradient of the log-distribution of x . $s_\theta(\mathbf{x}) \approx \nabla_{\mathbf{x}} \log p(\mathbf{x})$
- How can we use this to sample from $p(x)$?
- Langevin Equation by Paul Langevin runs for help:
 - The Equation provides a Monte Carlo Markov Chain procedure to sample from a distribution using only its score function. Specifically, it initializes the chain from an arbitrary prior distribution, and then iterates the following:

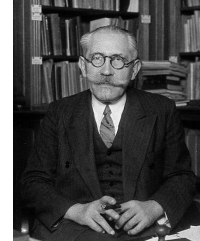
$$\mathbf{x}_{i+1} \leftarrow \mathbf{x}_i + \epsilon \nabla_{\mathbf{x}} \log p(\mathbf{x}) + \sqrt{2\epsilon} \mathbf{z}_i, \quad i = 0, 1, \dots, K,$$

Score-based Sampling

$$\frac{dx}{dt} = \frac{1}{2} \nabla \log p(x) + \sqrt{\epsilon} \eta(t)$$

Here, x is the state of the system, $\nabla \log p(x)$ is the score function, ϵ is a small step size, and $\eta(t)$ is a Gaussian noise term at each time step.

“There are days when I wish I did not have to wear a tie to work every day.”
Paul Langevin



- Here our latent space z is sampled from a $N(0,1)$. It is the stochastic element in Langevin's equation.

$$\mathbf{x}_{i+1} \leftarrow \mathbf{x}_i + \epsilon \nabla_{\mathbf{x}} \log p(\mathbf{x}) + \sqrt{2\epsilon} \mathbf{z}_i, \quad i = 0, 1, \dots, K,$$

- The idea is that, when $K \rightarrow \infty$, $\mathbf{x}_K$ obtained from the procedure in converges to a sample from $p(\mathbf{x})$.
- In practice, the error is negligible when is ϵ sufficiently small and K is sufficiently large.

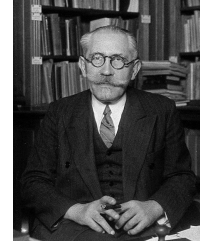


Score-based Sampling

$$\frac{dx}{dt} = \frac{1}{2} \nabla \log p(x) + \sqrt{\epsilon} \eta(t)$$

Here, x is the state of the system, $\nabla \log p(x)$ is the score function, ϵ is a small step size, and $\eta(t)$ is a Gaussian noise term at each time step.

“There are days when I wish I did not have to wear a tie to work every day.”
Paul Langevin



- Here our latent space z is sampled from a $N(0,1)$. It is the stochastic element in Langevin's equation.

$$\mathbf{x}_{i+1} \leftarrow \mathbf{x}_i + \epsilon \nabla_{\mathbf{x}} \log p(\mathbf{x}) + \sqrt{2\epsilon} \mathbf{z}_i, \quad i = 0, 1, \dots, K,$$

- The idea is that, when $K \rightarrow \infty$, $\mathbf{x}_K$ obtained from the procedure converges to a sample from $p(\mathbf{x})$.
- In practice, the error is negligible when ϵ is sufficiently small and K is sufficiently large.

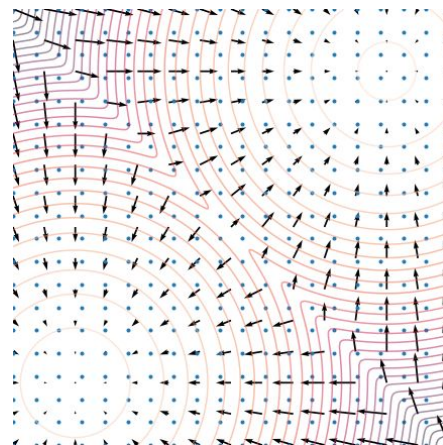


figure from Yang Song's blog

Score-based Diffusion Limitations

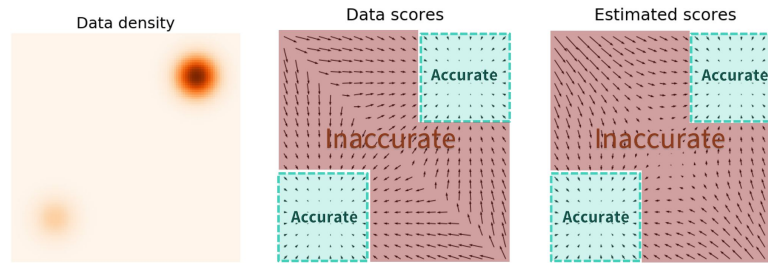
- The key challenge is the fact that the estimated score functions are inaccurate in low density regions, where few data points are available for computing the score matching objective.
- This is expected as score matching minimizes the Fisher divergence:

$$\mathbb{E}_{p(\mathbf{x})}[\|\nabla_{\mathbf{x}} \log p(\mathbf{x}) - \mathbf{s}_{\theta}(\mathbf{x})\|_2^2] = \int p(\mathbf{x}) \|\nabla_{\mathbf{x}} \log p(\mathbf{x}) - \mathbf{s}_{\theta}(\mathbf{x})\|_2^2 d\mathbf{x}.$$

- Since the ℓ_2 differences between the true data score function and score-based model are weighted by $p(\mathbf{x})$, they are largely ignored in low density regions where $p(\mathbf{x})$ is small.

Score-based Diffusion Limitations

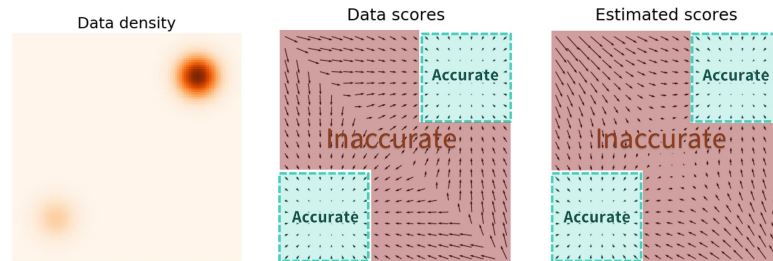
- This behavior can lead to subpar results, as illustrated by the figure below:



- When sampling with Langevin dynamics, our initial sample is highly likely in low density regions when data reside in a high dimensional space.
- Therefore, having an inaccurate score-based model will derail Langevin dynamics from the very beginning of the procedure, preventing it from generating high quality samples that are representative of the data.

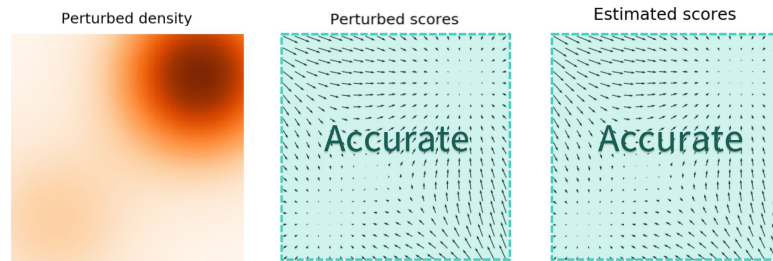
How to solve low $p(x)$ problem?

- How can we bypass the difficulty of accurate score estimation in regions of low data density?
- The idea is to perturb data points with noise and train score-based models on the noisy data points instead.
- When the noise magnitude is sufficiently large, it can populate low data density regions to improve the accuracy of estimated scores.



How to solve low $p(x)$ problem?

- How can we bypass the difficulty of accurate score estimation in regions of low data density?
- The idea is to perturb data points with noise and train score-based models on the noisy data points instead.
- When the noise magnitude is sufficiently large, it can populate low data density regions to improve the accuracy of estimated scores.



but how?

How to solve low $p(x)$ problem

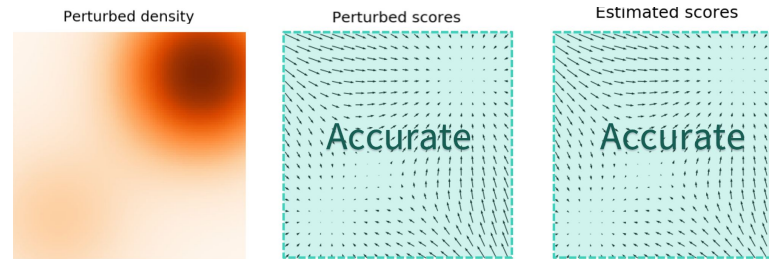
- How can we bypass the difficulty of accurate regions of low data density?
- The idea is to perturb data points with noise and train models on the noisy data points instead.
- When the noise magnitude is sufficiently large, the model can learn to generate data in low data density regions to improve the accuracy.

Yang Song
Stanford University
yangsong@cs.stanford.edu

Stefano Ermon
Stanford University
ermon@cs.stanford.edu

Abstract

We introduce a new generative model where samples are produced via Langevin dynamics using gradients of the data distribution estimated with score matching. Because gradients can be ill-defined and hard to estimate when the data resides on low-dimensional manifolds, we perturb the data with different levels of Gaussian noise, and jointly estimate the corresponding scores, *i.e.*, the vector fields of gradients of the perturbed data distribution for all noise levels. For sampling, we propose an annealed Langevin dynamics where we use gradients corresponding to gradually decreasing noise levels as the sampling process gets closer to the data manifold. Our framework allows flexible model architectures, requires no sampling during training or the use of adversarial methods, and provides a learning objective that can be used for principled model comparisons. Our models produce samples comparable to GANs on MNIST, CelebA and CIFAR-10 datasets, achieving a new state-of-the-art inception score of 8.87 on CIFAR-10. Additionally, we demonstrate that our models learn effective representations via image inpainting experiments.



but how?

How to solve low $p(x)$ problem

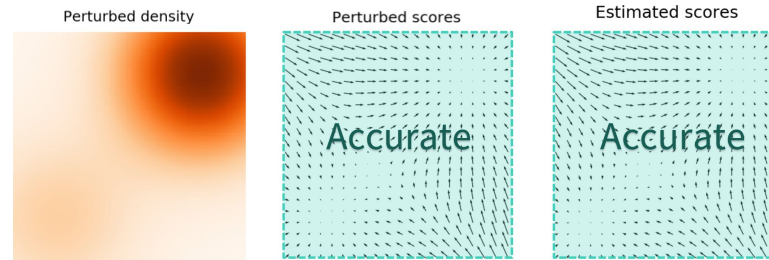
- How can we bypass the difficulty of accurate regions of low data density?
- The idea is to perturb data points with noise models on the noisy data points instead.
- When the noise magnitude is sufficiently large, data density regions to improve the accuracy

Yang Song
Computer Science Department
Stanford University
yangsong@cs.stanford.edu

Stefano Ermon
Computer Science Department
Stanford University
ermon@cs.stanford.edu

Abstract

Score-based generative models can produce high quality image samples comparable to GANs, without requiring adversarial optimization. However, existing training procedures are limited to images of low resolution (typically below 32×32), and can be unstable under some settings. We provide a new theoretical analysis of learning and sampling from score-based models in high dimensional spaces, explaining existing failure modes and motivating new solutions that generalize across datasets. To enhance stability, we also propose to maintain an exponential moving average of model weights. With these improvements, we can scale score-based generative models to various image datasets, with diverse resolutions ranging from 64×64 to 256×256 . Our score-based models can generate high-fidelity samples that rival best-in-class GANs on various image datasets, including CelebA, FFHQ, and several LSUN categories.



but how?

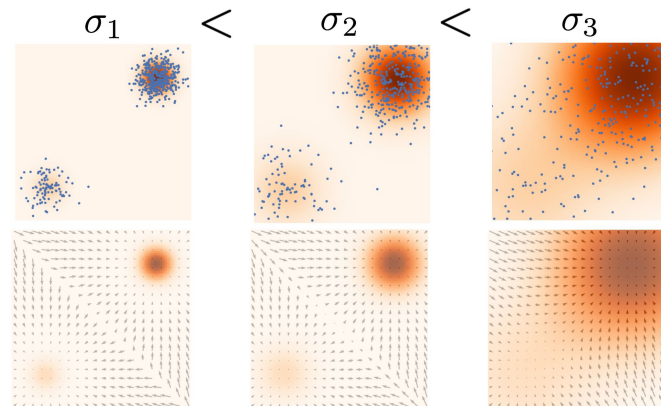
How to solve low $p(\mathbf{x})$ problem?

- Song proposes **Noise Conditional Score-Based Model** (also called a Noise Conditional Score Network, or NCSN, when parameterized with a neural network) to solve this problem.
- They use multiple scales of noise perturbations simultaneously.
 - perturb the data with isotropic (same variance in all dimensions) Gaussian noise,
 - And let there be a total of L increasing standard deviations : $\sigma_1 < \sigma_2 < \dots < \sigma_L$
 - So we have family of isotropic Gaussian noise: $\mathcal{N}(0, \sigma_i^2 I), i = 1, 2, \dots, L$
 - noise-perturbed version of the distribution $p(\mathbf{x})$ is: $p_{\sigma_i}(\mathbf{x}) = \int p(\mathbf{y}) \mathcal{N}(\mathbf{x}; \mathbf{y}, \sigma_i^2 I) d\mathbf{y}$.
 - The Noise Conditional Score-Based Model is:
 $\mathbf{s}_\theta(\mathbf{x}, i) \approx \nabla_{\mathbf{x}} \log p_{\sigma_i}(\mathbf{x})$ for all $i = 1, 2, \dots, L$.

How to solve low $p(\mathbf{x})$ problem?

- Song proposes **Noise Conditional Score-Based Model** (also called a Noise Conditional Score Network, or NCSN, when parameterized with a neural network) to solve this problem.
- They use multiple scales of noise perturbations simultaneously.
 - The training objective for is a weighted sum of Fisher divergences for all noise scales:

$$\sum_{i=1}^L \lambda(i) \mathbb{E}_{p_{\sigma_i}(\mathbf{x})} [\|\nabla_{\mathbf{x}} \log p_{\sigma_i}(\mathbf{x}) - \mathbf{s}_{\theta}(\mathbf{x}, i)\|_2^2],$$



How to sample the noise conditioned model?

- After training the noise-conditional score-based model we can produce samples from it by running Langevin dynamics for $i = L, L-1, \dots, 1$.
- This method is called **annealed Langevin dynamics** since the noise scale decreases (anneals) gradually over time.

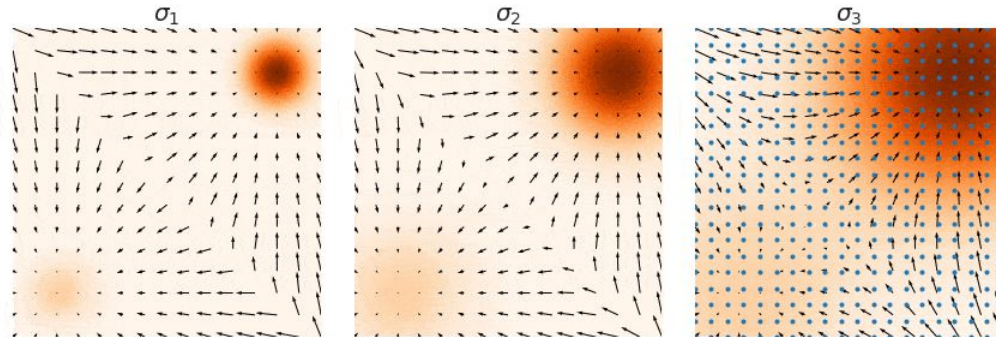
Generative Modeling by Estimating Gradients of the Data Distribution

Yang Song
Stanford University
yangsong@stanford.edu

Stefano Ermon
Stanford University
ermon@stanford.edu

Abstract

We introduce a new generative model where samples are produced via Langevin dynamics using gradients of the data distribution estimated with score matching. Because gradients can be estimated and backpropagated when the data reside on low-dimensional manifolds, we perturb the data with different levels of Gaussian noise, and jointly estimate the corresponding scores, i.e., the vector fields of gradients of the perturbed data distribution (or at noise levels). For sampling, we propose an annealed Langevin dynamics where we use gradients corresponding to gradually decreasing noise levels as the sampling process gets closer to the data manifold. Our framework allows flexible model architectures, requires no sampling during training or the use of adversarial methods, and provides a learning objective that can be used for principled model comparison. Our models produce samples comparable to GANs on MNIST, CIFAR-10, and CIFAR-100 datasets, achieving a new state-of-the-art inception score of 8.5 on CIFAR-10. Additionally, we demonstrate that our models learn effective representations via image separating experiments.



What will we do next week?

- Week 7: TimesNet-Gen: How to make light-weight model generative by Barış Yılmaz
- Week 8: In classroom classical exam